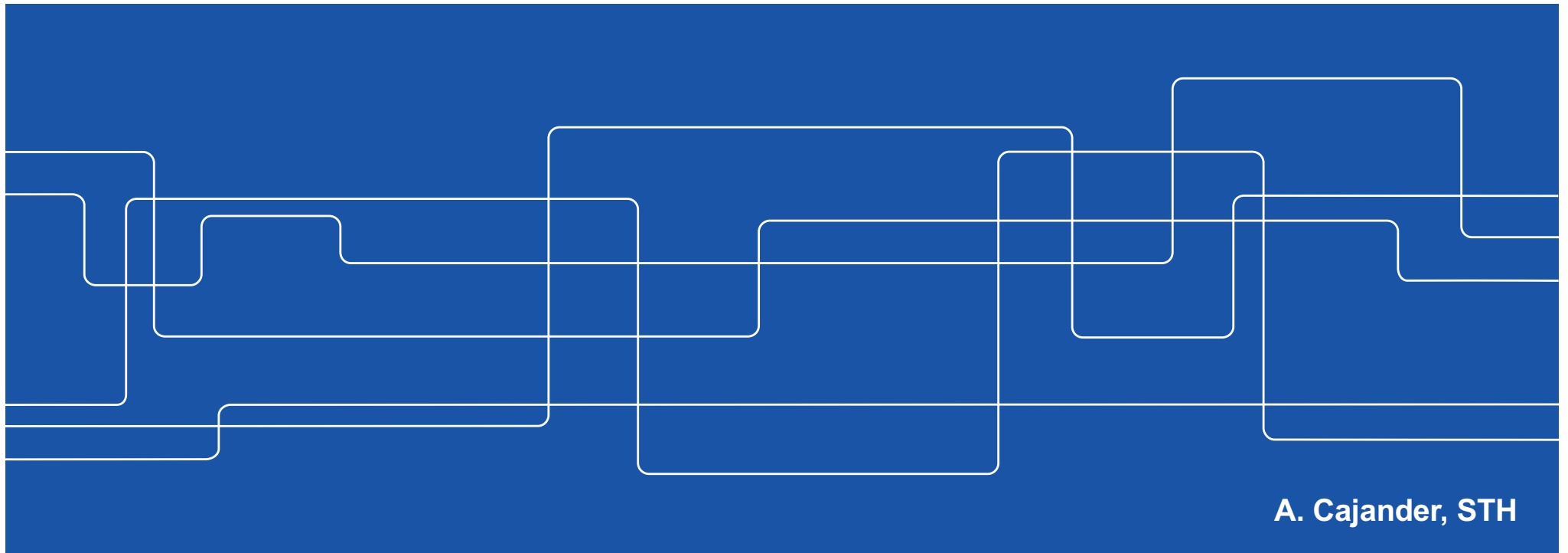




HE1025 Operativsystem

F3: Virtualisering Minnet - Segmentation



A. Cajander, STH



Kursen...

F1: V#1 Processorn.

Ö1: V#2 c Pekare...

F2: V#3 Schemaläggning

L1: V#4 FreeRTOS

F3: V#5 Segmentation

Ö2: Eget arbete

F4: V#7 Paging

L2: V#8 Paging

F5: C#1 Locks & CV

Ö3: C#2 LINUX

F6: C#3 Semaphores

L3: C#4 Barberare

F7: P#1 Storage

Ö4: P#2 Buffer

F8: P#3 Communication

L4: P#4 Client/Server

Kap (1) 2..4 & 6

Kap 14 & c-boken

Kap 7..9,(10-11), RTOS

Lab #1 Handledning

Kap 12..13, 15..17

Kap 18-22, (23-24)

Lab #2 Handledning

Kap (25) 26..27 & 28..29.1

Kap 5 (39)

Kap 30..33, (34)

Lab #3 Handledning

Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)

t.b.d

Kap (47, 39) & 48, (49..51)

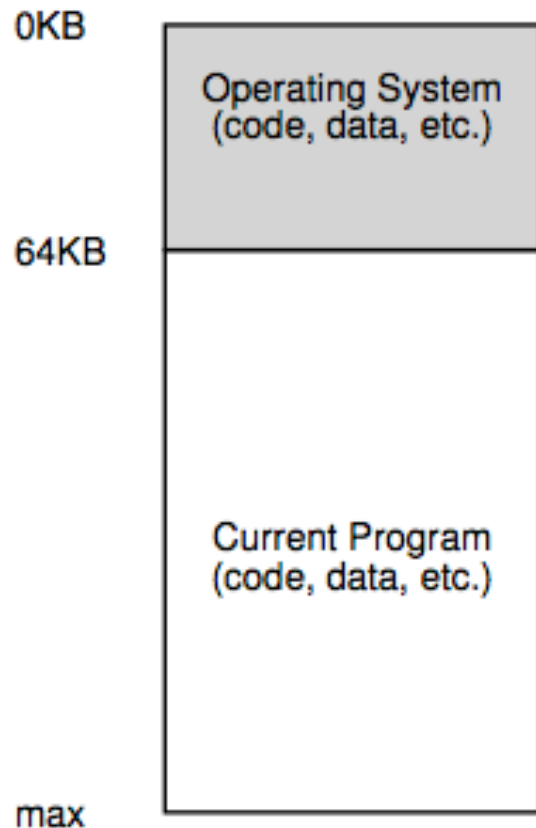
Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Närvaro L1..4



Virtualisering av minnet...



Så snart det fanns en Boot-loader/monitor/os så uppstod problemet att det laddade programmet sällan började på adress 0 i minnet...

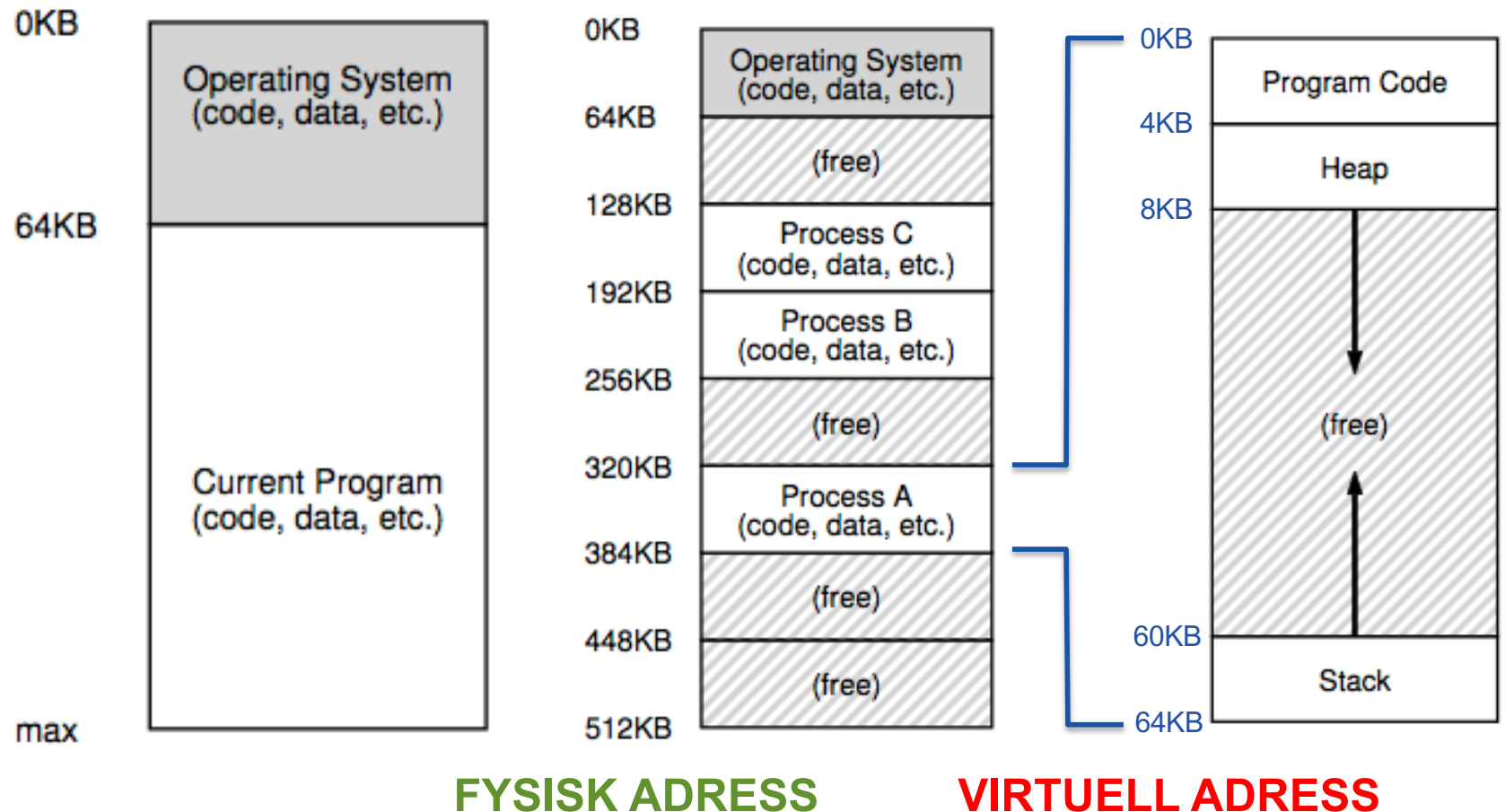
...till en början, då programmen alltid var skrivna i assembler, så räckte det med att programmeraren lade till en:

org 0x<start adress>

Så fixade assemblern så att koden blev rätt! **(Men programmet kunde bara köras på ett fast ställe i minnet.)**

(I princip alla assemblerspråk har ett origin-direktiv som används för att placera kod/data i förhållande till varandra, ej säkert fysiskt address.)

Virtualisering av minnet...



Det skulle vara mycket bättre om datorns minne också var virtualiserat...
...så att varje process hade illusionen av ett helt eget minne!



Virtualisering av minnet...

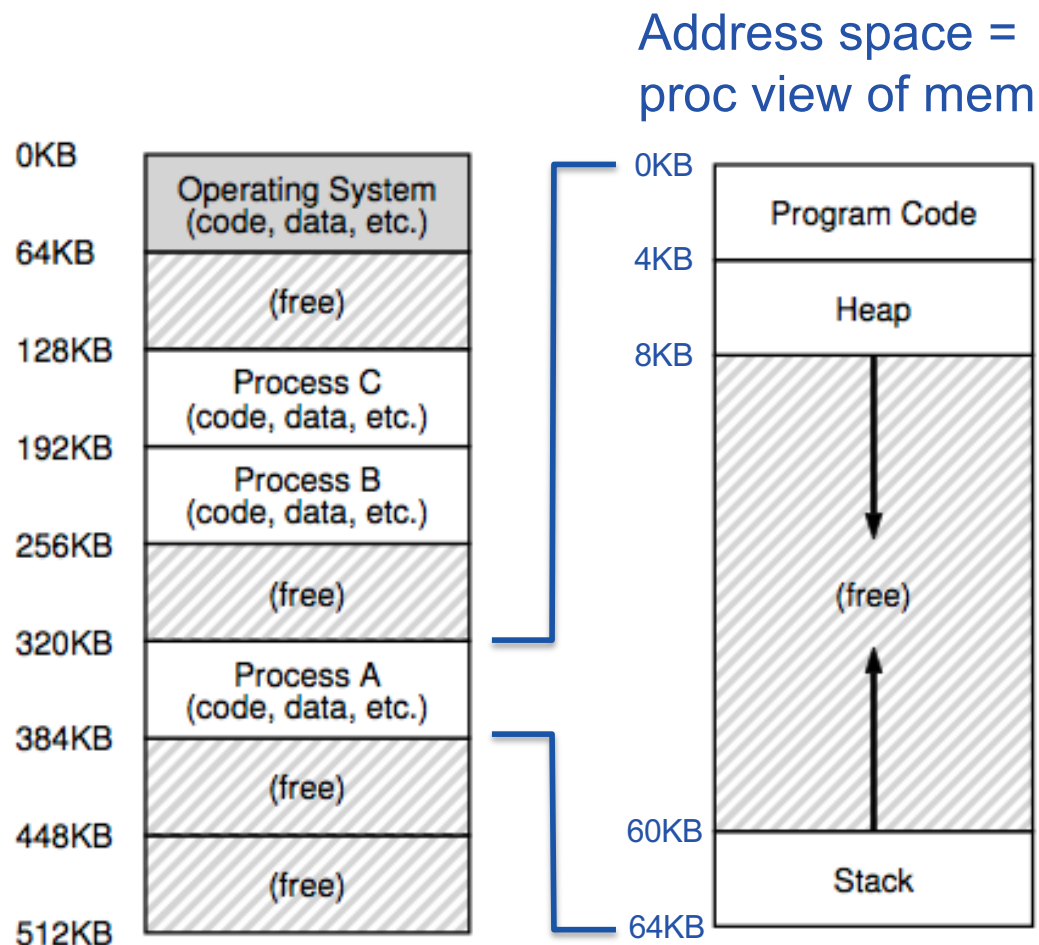
Konverteringen bör vara:

Transparant

Effektiv

Protection/Isolation

Robust



FYSISK ADRESS

VIRTUELL ADRESS

Varje adress du någonsin kommer att "se" är virtuell!



Virtualisering av minnet...

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    printf("location of code   : %p\n", (void *) main);
    printf("location of heap   : %p\n", (void *) malloc(1));
    int x = 3;
    printf("location of stack  : %p\n", (void *) &x);
    return x;
}
```

When run on a 64-bit Mac, we get the following output:

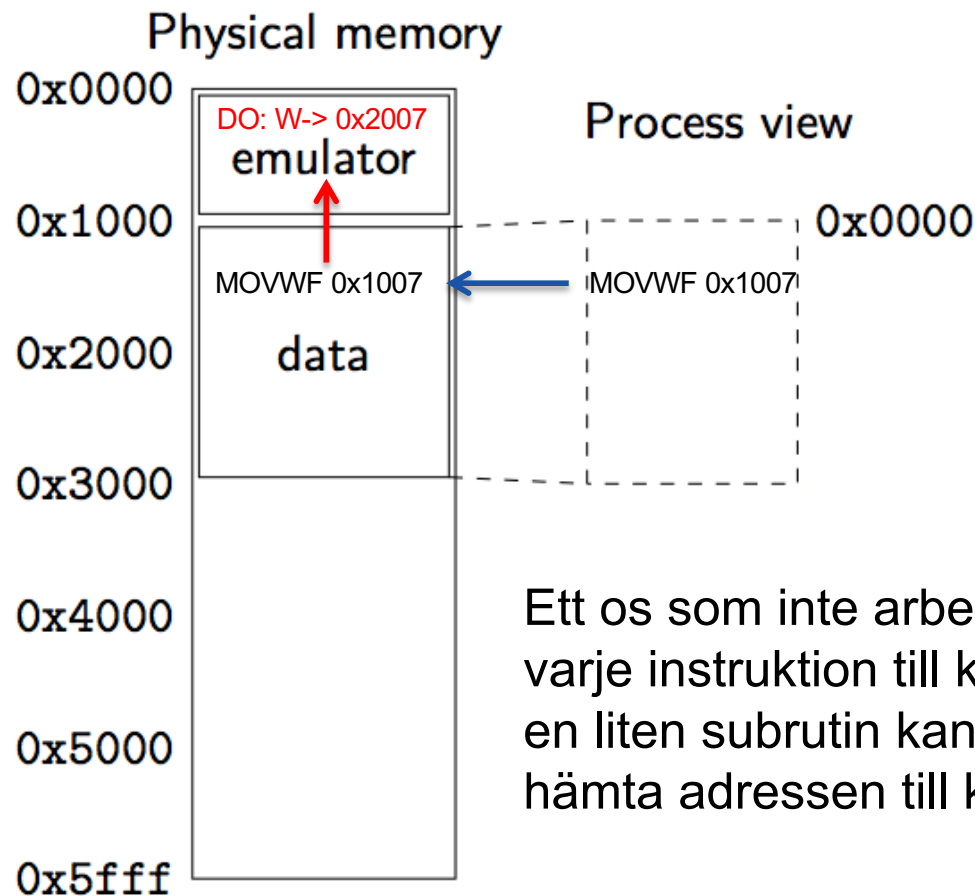
```
location of code   : 0x1095afe50
location of heap   : 0x1096008c0
location of stack  : 0x7fff691aea64
```

Fysiskt minne 64-bitar > max 16 EByte
Virtuellt minne 47-bitar > max 128 TByte

4 Gbyte?!

...via emulering?

(Försök #1)



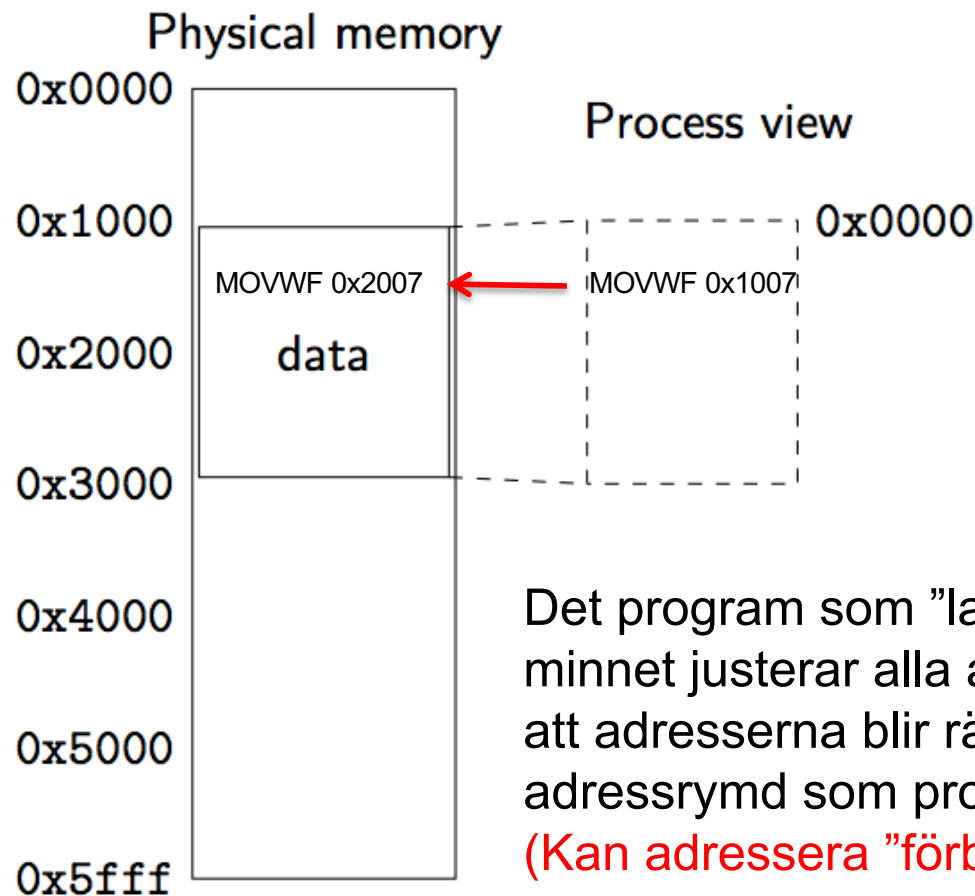
Ett os som inte arbetar enligt LDE utan hämtar varje instruktion till kärnan för att utföra den som en liten subrutin kan OS:et lätt själv justera den hämta adressen till korrekt fysiskt adress...

...både exekveringen av instruktionen och korrigeringen av adressen kommer att ta tid. Enkel implementation men mycket ineffektiv!



...static relocation?

(Försök #2)

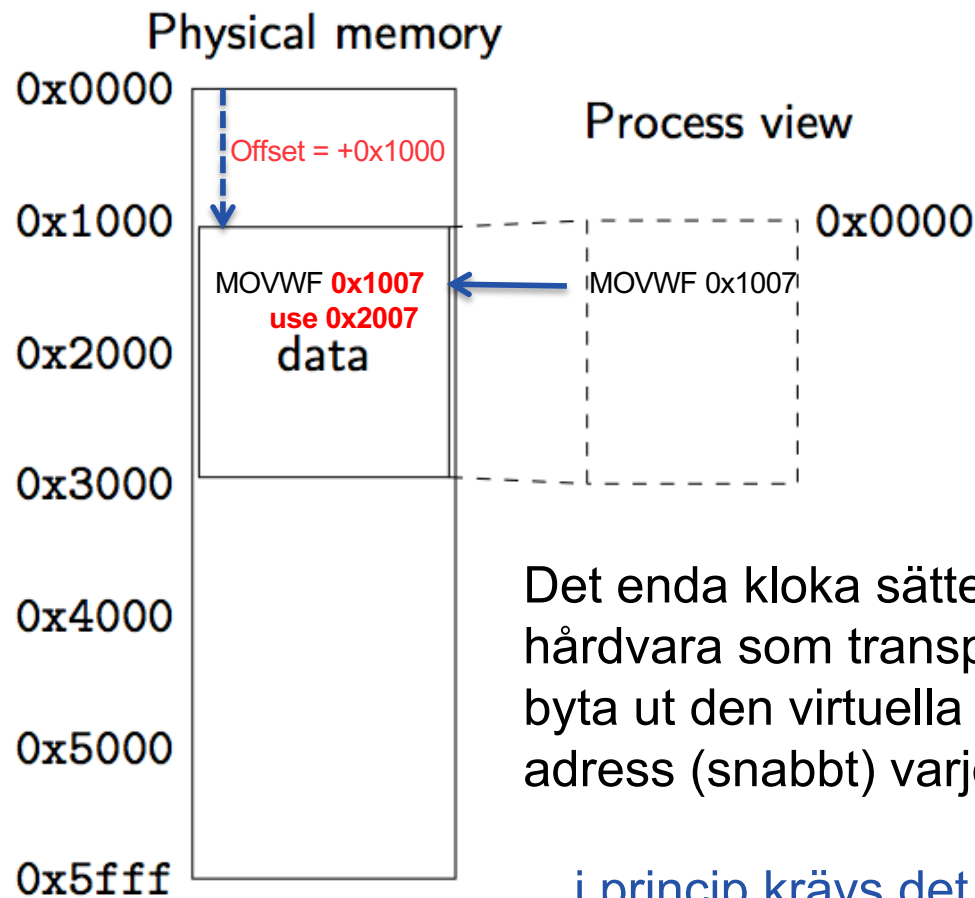


Det program som "laddar in" processens kod i minnet justerar alla adresser i koden (hjälp!) så att adresserna blir rätt, givet den fysiska adressrymd som processen tilldelats!

(Kan adressera "förbjudna" minnesceller)

...Användes på -50 & -60-talet, inte längre!

...dynamic (HW) relocation? (Försök #3)

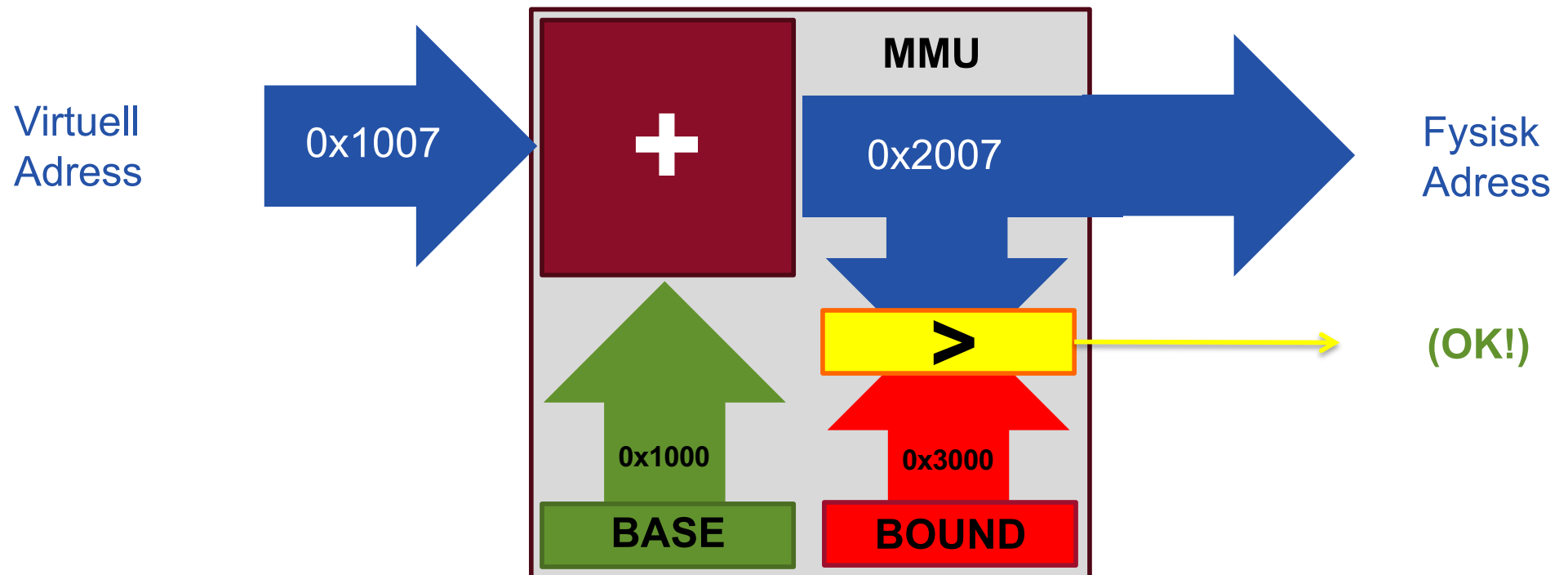


Det enda kloka sättet är om det finns extra hårdvara som transparent för en LDE cpu kan byta ut den virtuella adressen mot rätt fysisk adress (snabbt) varje gång det behövs...

...i princip krävs det bara att en offset adderas...

...en sådan extra hårdvara benämns vanligtvis memory management unit (MMU) och finns i dag ofta inbyggd i samma chip som processorn!

#1: Base & bound modellen!



...Base & Bound-registren måste givetvis innehålla rätt värden för den process som för stunden kör, en till uppgift för schemaläggaren...

...M.a.o. så måste base/bound-värdena lagras i respektive proc PCB, men det medger också att minnet kan "flyttas" vid varje context switch!



Från ett OS perspektiv...

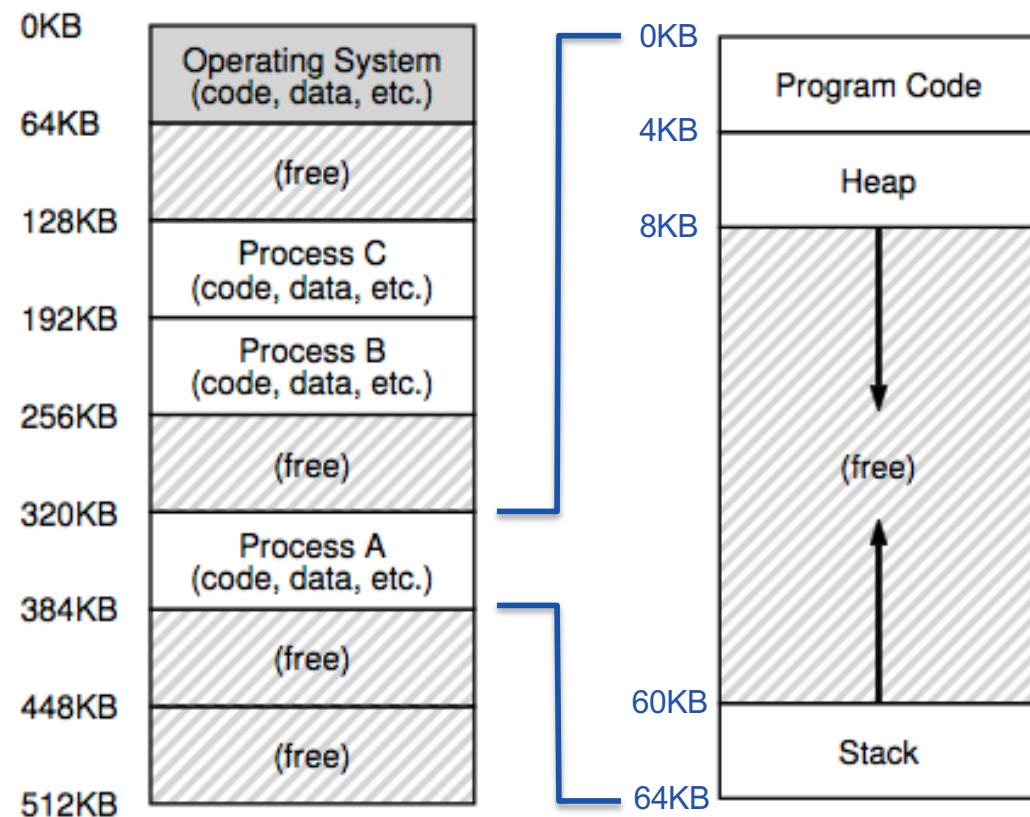
¹ Processerna har lika stora minnesblock så jobbet blir rätt enkelt

OS:et måste känna till vilket minne som är ledigt¹ (free list)

Ny process tilldelas fritt minne, kärninst. hanterar base/bound registren när process får köra!

En otroligt stor del av processens tilldelade² adressrymd använder inte processen?!

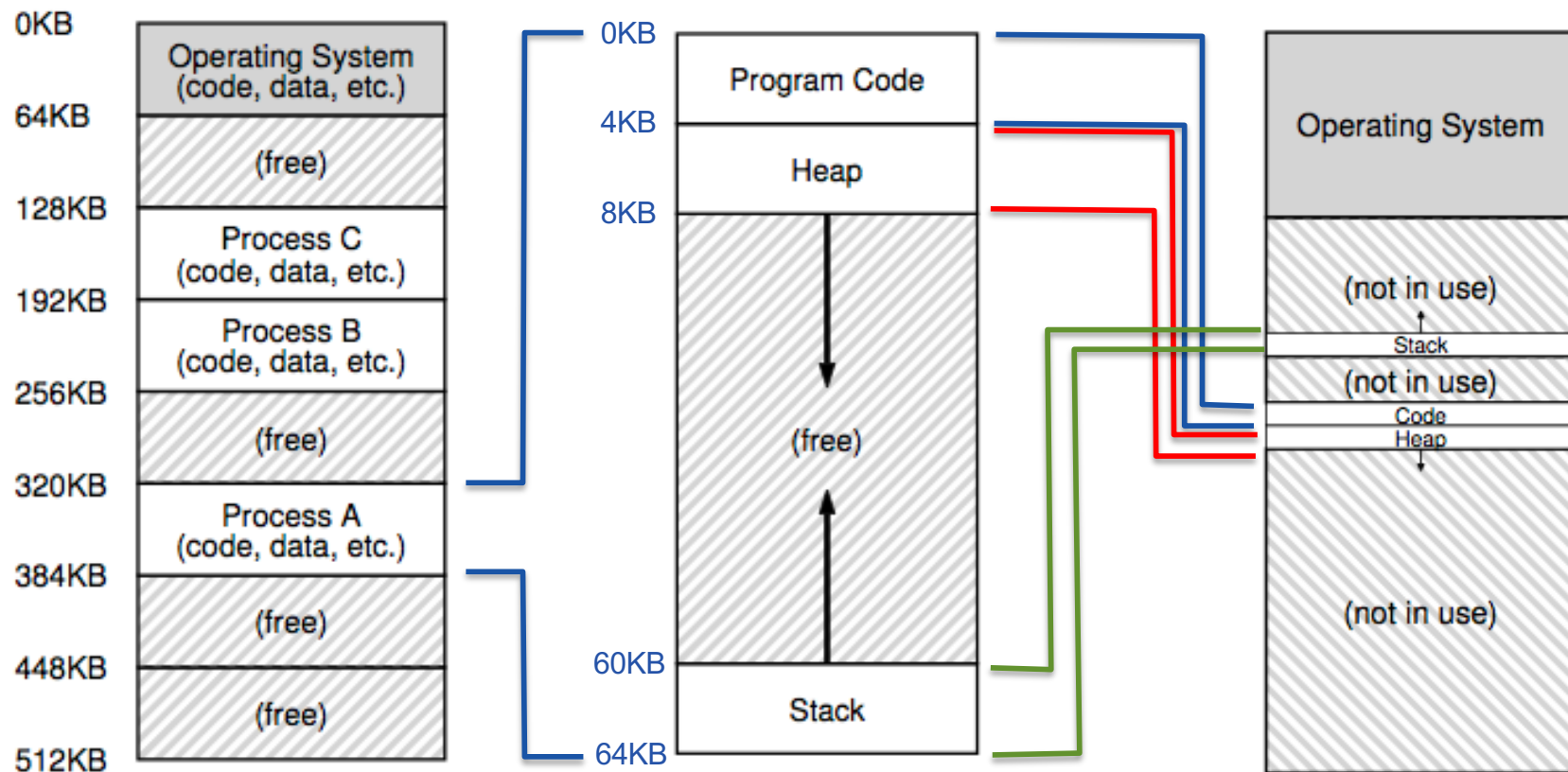
När en process är klar måste dess minne återlämnas till det lediga minnet (free list)



(Det lediga minnet kan då bli fragmenterat = extern fragmentering, ett senare krux!)

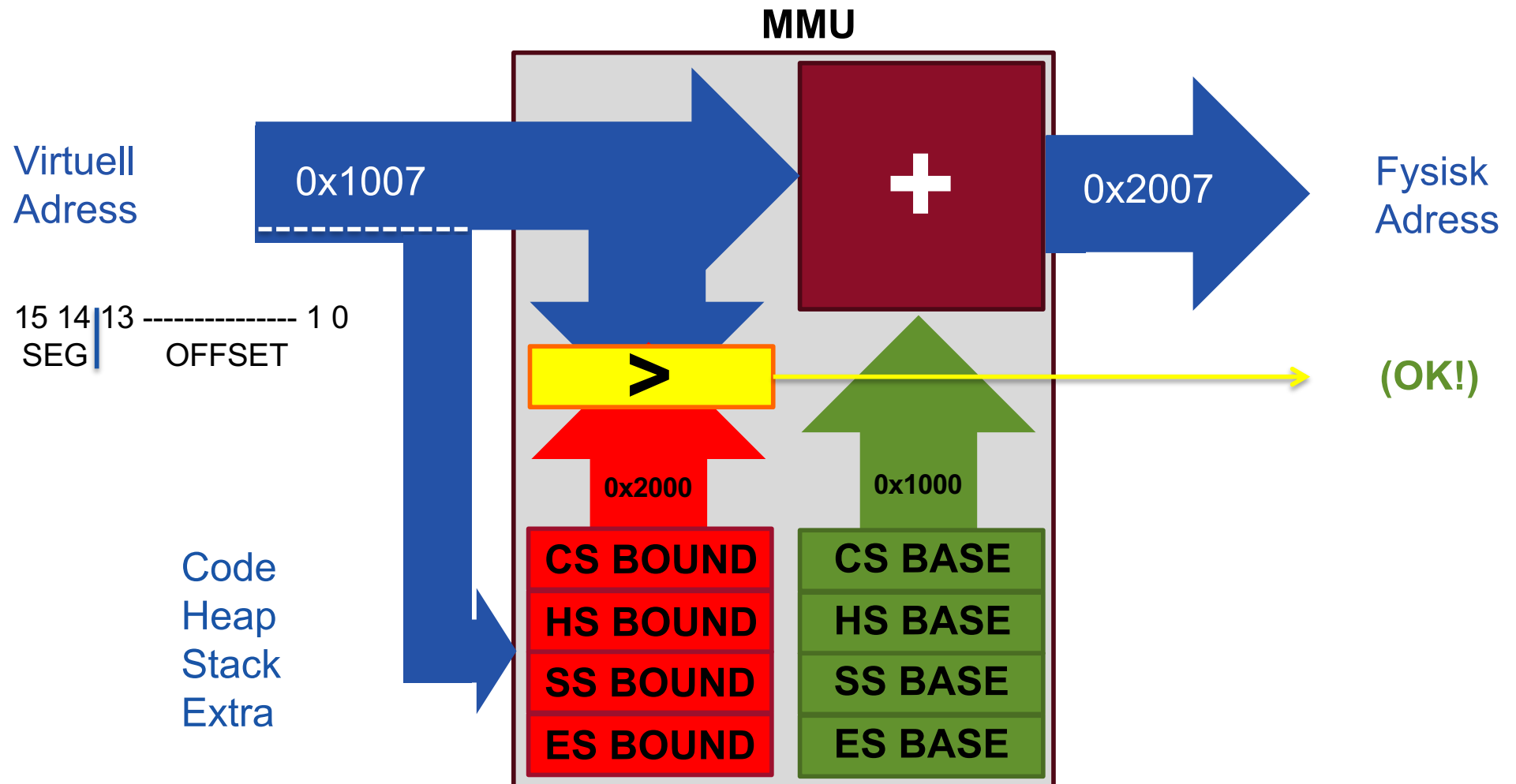
² Problemet benämns intern fragmentering och måste hanteras...
(Tilldelat minnesblock är större än vad som används/efterfrågats)

#2: Segment modellen!



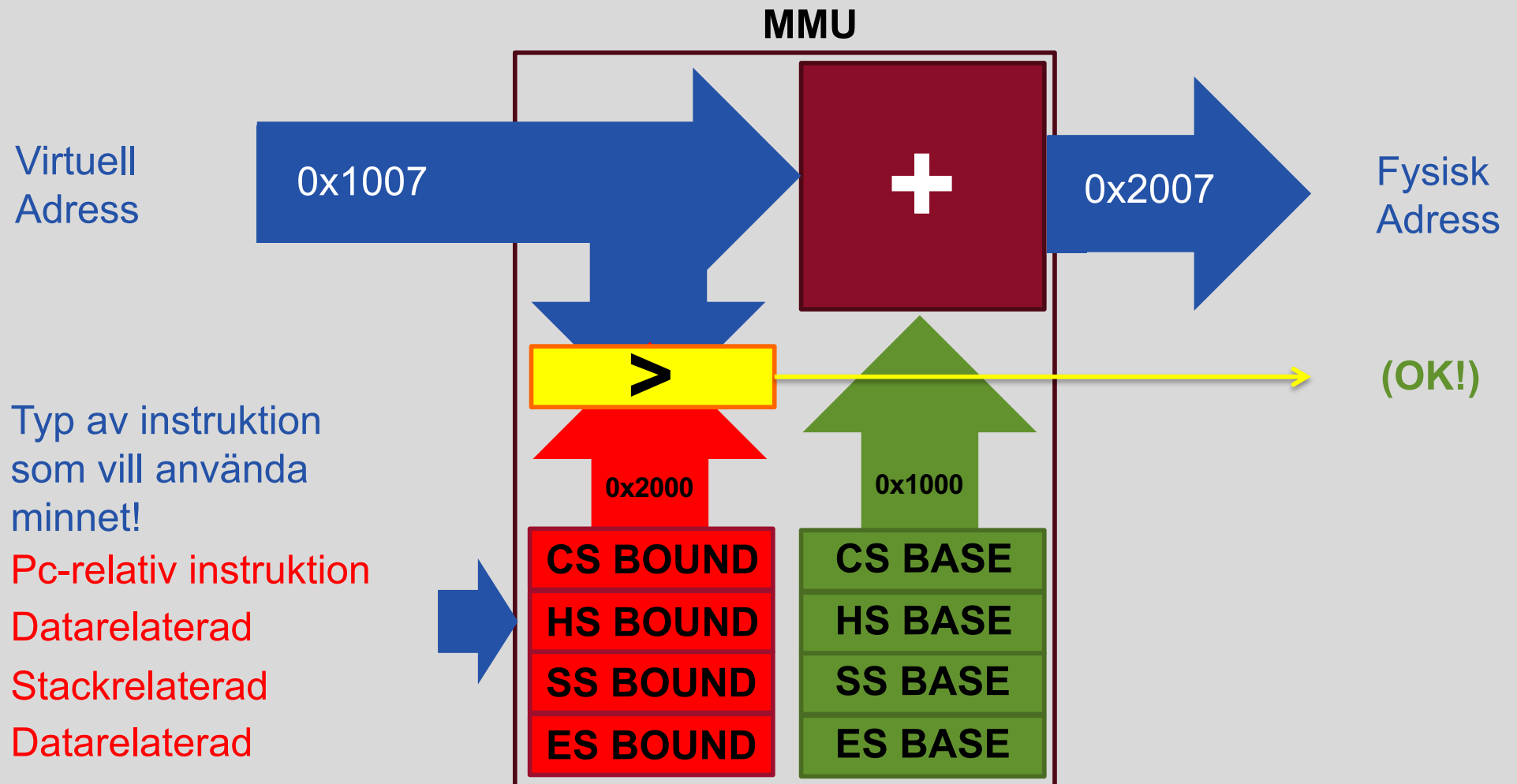
Processens olika segment kan allokeras fritt i det fysiska minnet...

#2: Segment modellen!



...t.ex. så används de 2 msb till att bestämma vilket segment som avses, resterande bitar blir en offset i segmentet (gör bound-testet enklare)...

#2: Segment modellen!



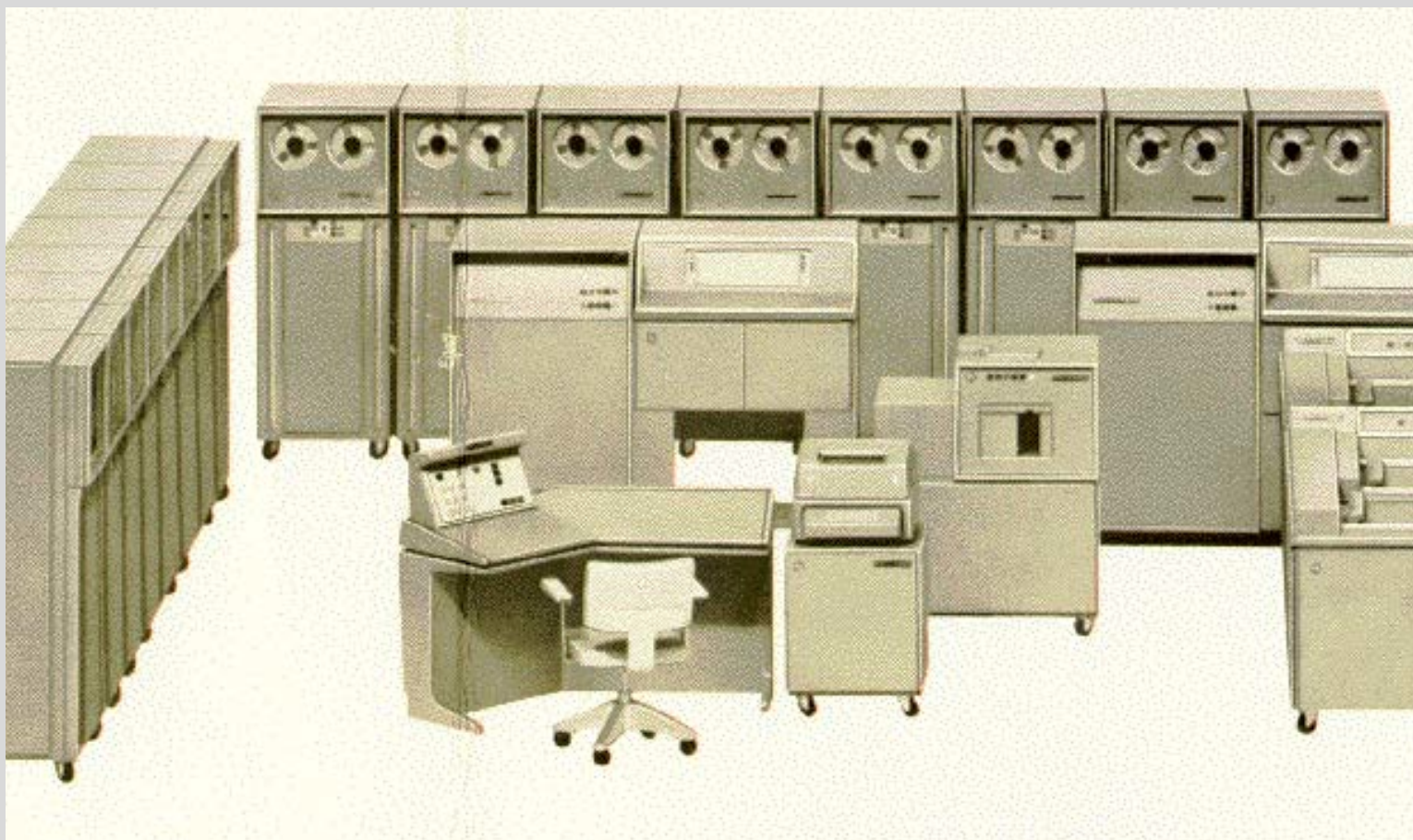


I verkligheten krävs lite till...

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write

...stacken växer mot lägre adresser, dela t.ex. kodsegment...

Burroughs B5000, 1961, Algol



Första datorn att använda segment...

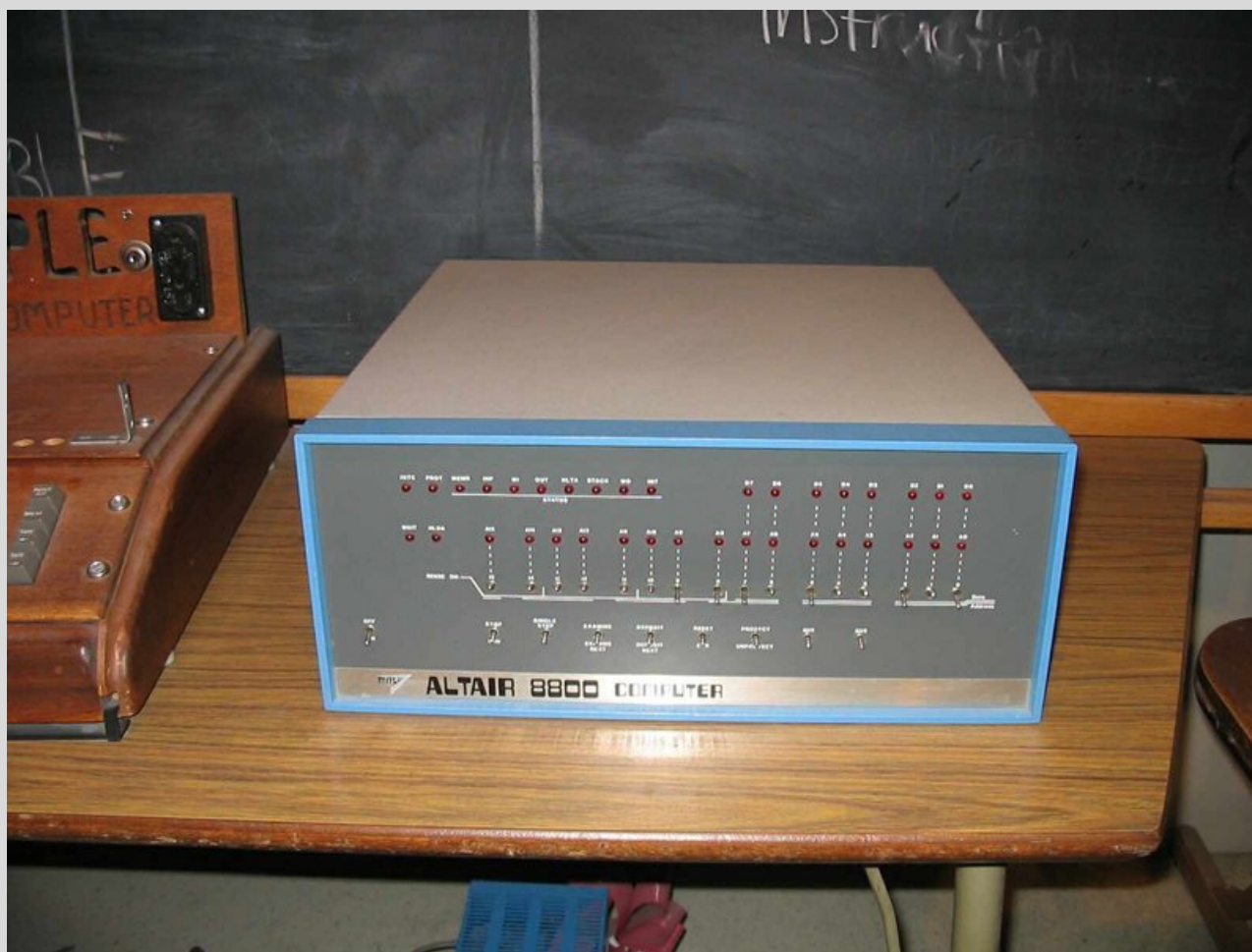
Donald Knuth arbetade med systemet.
Nu professor Emeritus Stanford University
& utsågs till hedersdoktor vid KTH -91!

Digital DEC-10, 1966



Använde 2 segment/process, kod respektive data.

Intel 8080, 1972



Världens första mikrodator/persondator...
...programmerades i BASIC från nystartade företaget Microsoft...

Legend:

- IMP
- △ PLURIBUS IMP
- TIP
- ~ SATELLITE CIRCUIT

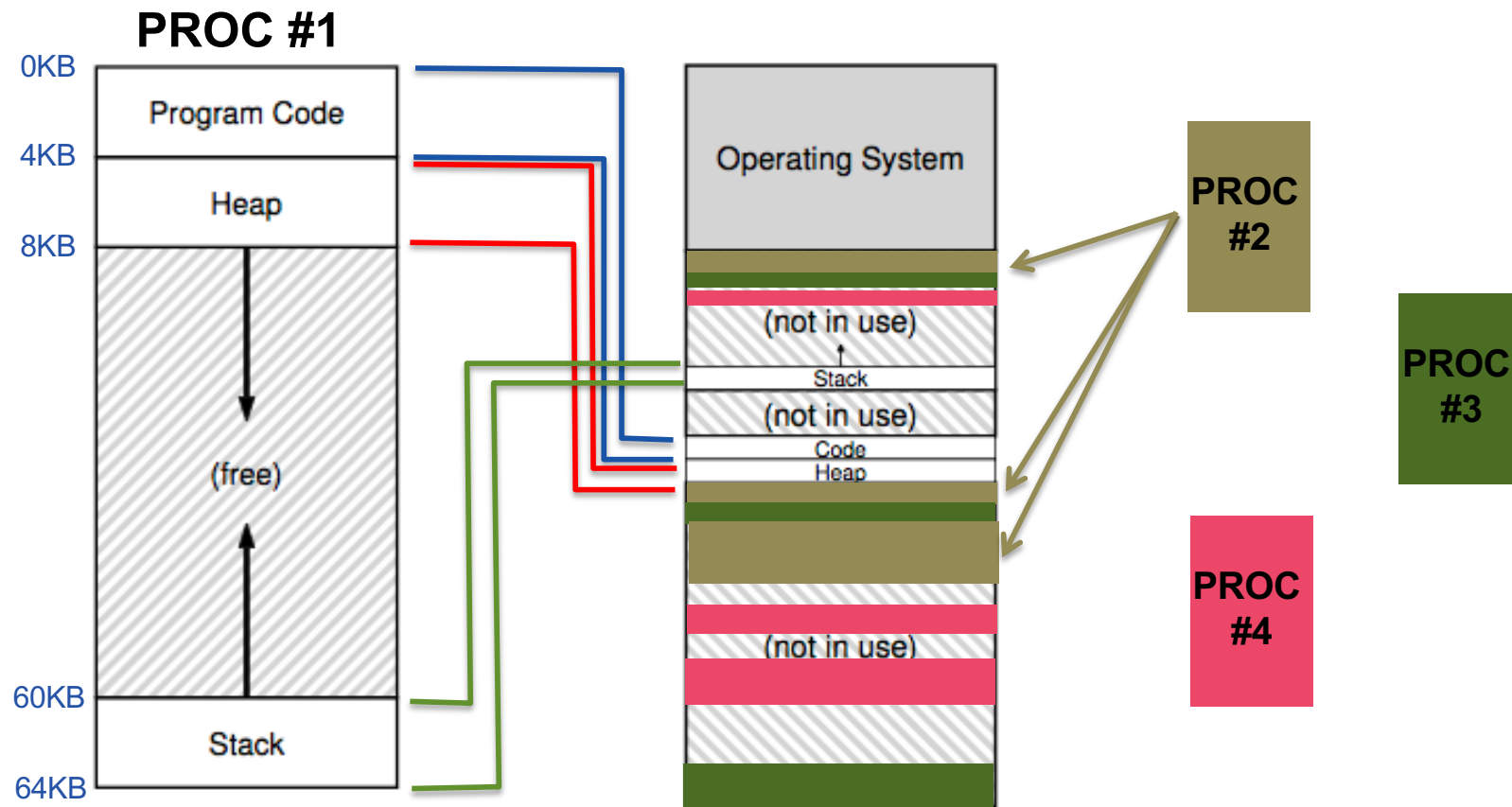
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
84

Intel 8086, 1978, Bakåtkomp. med 8080...



...fyra segment: code, data, stack & extra,
basen för alla x86 processorer!

MEN: Segment är olika stora...

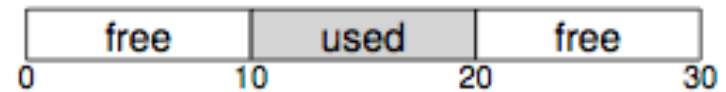


...så nu krävs något verkligen lurigt för att hantera det fria minnet!

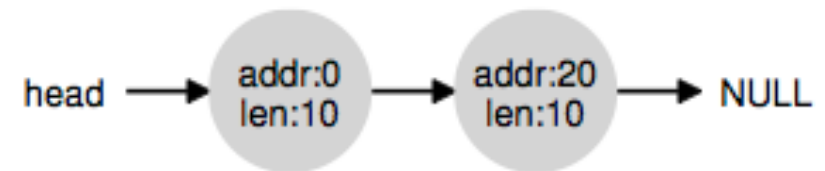


Splitting & Coaleasing

Ett litet 30Byte RAM minne...



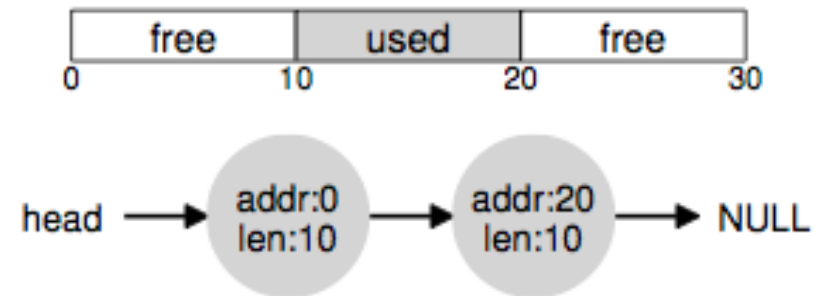
OS:ets Free-list...





Splitting & Coaleasing

Ett litet 30Byte RAM minne...



OS:ets Free-list...

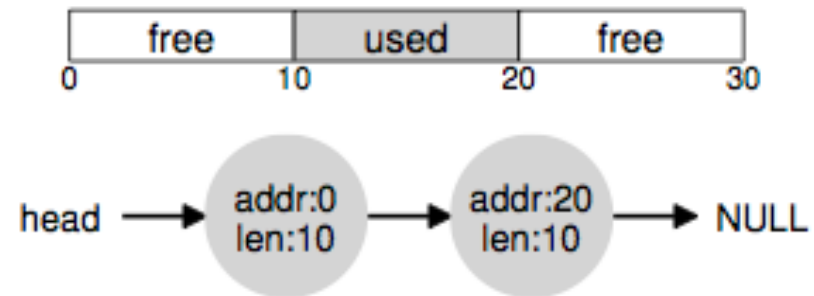
OS:et behöver 20 Bytes?

Omöjligt!



Splitting & Coaleasing

Ett litet 30Byte RAM minne...



OS:et behöver 20 Bytes?

Omöjligt!

OS:et behöver 10 Bytes?

Enkelt, returnera 0 eller 20!

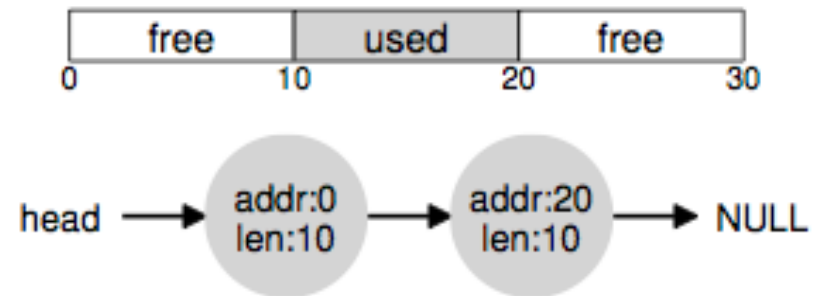
OS:et behöver 1 Byte?

Det är dumt att dela ut ett helt block, någon av blocken måste delas (splitting), men vad är bästa sättet att välja var?



Splitting & Coaleasing

Ett litet 30Byte RAM minne...



OS:ets Free-list...

OS:et behöver 20 Bytes?

Omöjligt!

OS:et behöver 10 Bytes?

Enkelt, returnera 0 eller 20!

OS:et behöver 1 Byte?

Det är dumt att dela ut ett helt block, någon av blocken måste delas (splitting), men vad är bästa sättet att välja var?

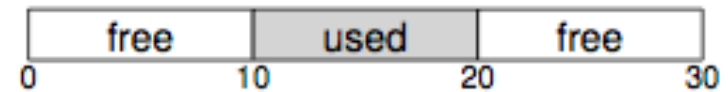
Vi återkommer till det, säg att vi tar 1 Byte i andra blocket...



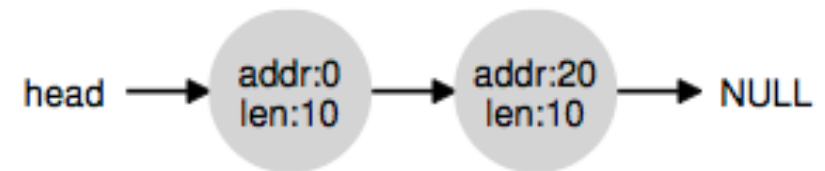


Splitting & Coaleasing

Ett litet 30Byte RAM minne...



OS:ets Free-list...

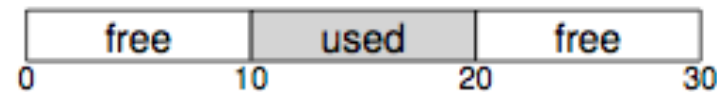


Det använda minnet lämnas tillbaks?!



Splitting & Coaleasing

Ett litet 30Byte RAM minne...



OS:ets Free-list...

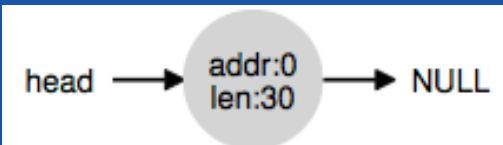


Det använda minnet lämnas tillbaks?!



Det här blev helt galet, uppenbart finns 30 Byte ledigt men alla önskemål om mer än 10 Byte kommer att nekas...

...när minne återlämnas måste os:et kunna slå samman (coaleasc) block, om det går, till större!





Spåra storleken på utdelat minne?!

```
pltem = malloc( sizeof(Item) );    // Allocate space for a new Item!  
*  
*  
free( pltem );                    // Return space to OS!
```

Hur vet OS:et hur många byte som returneras när free() bara har ett argument som är adressen till den första (enda?) byten i blocket?!



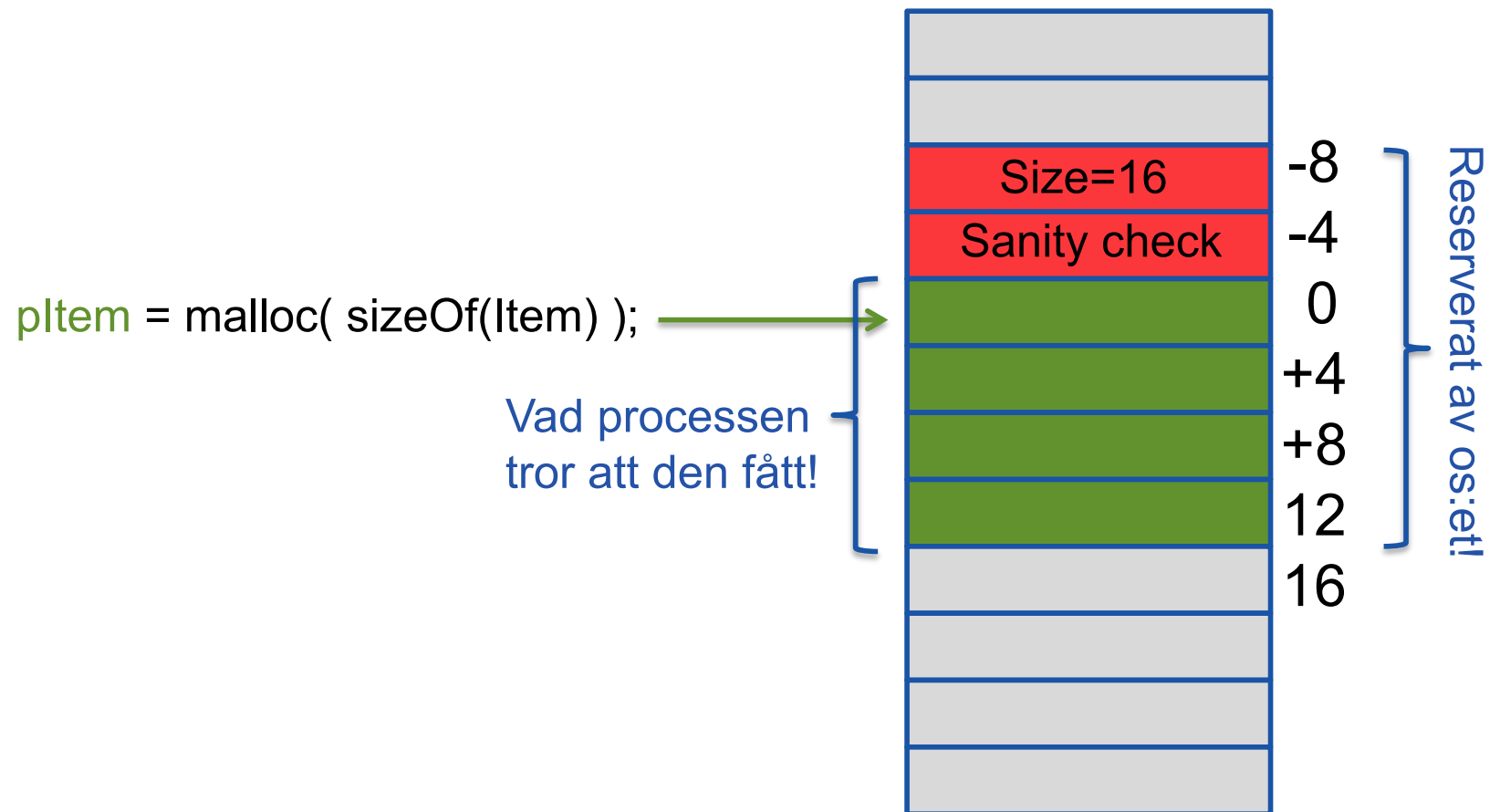
Spåra storleken på utdelat minne?!

```
pltem = malloc( sizeof(Item) );      // Allocate space for a new Item!  
*  
*  
free( pltem );                      // Return space to OS!
```

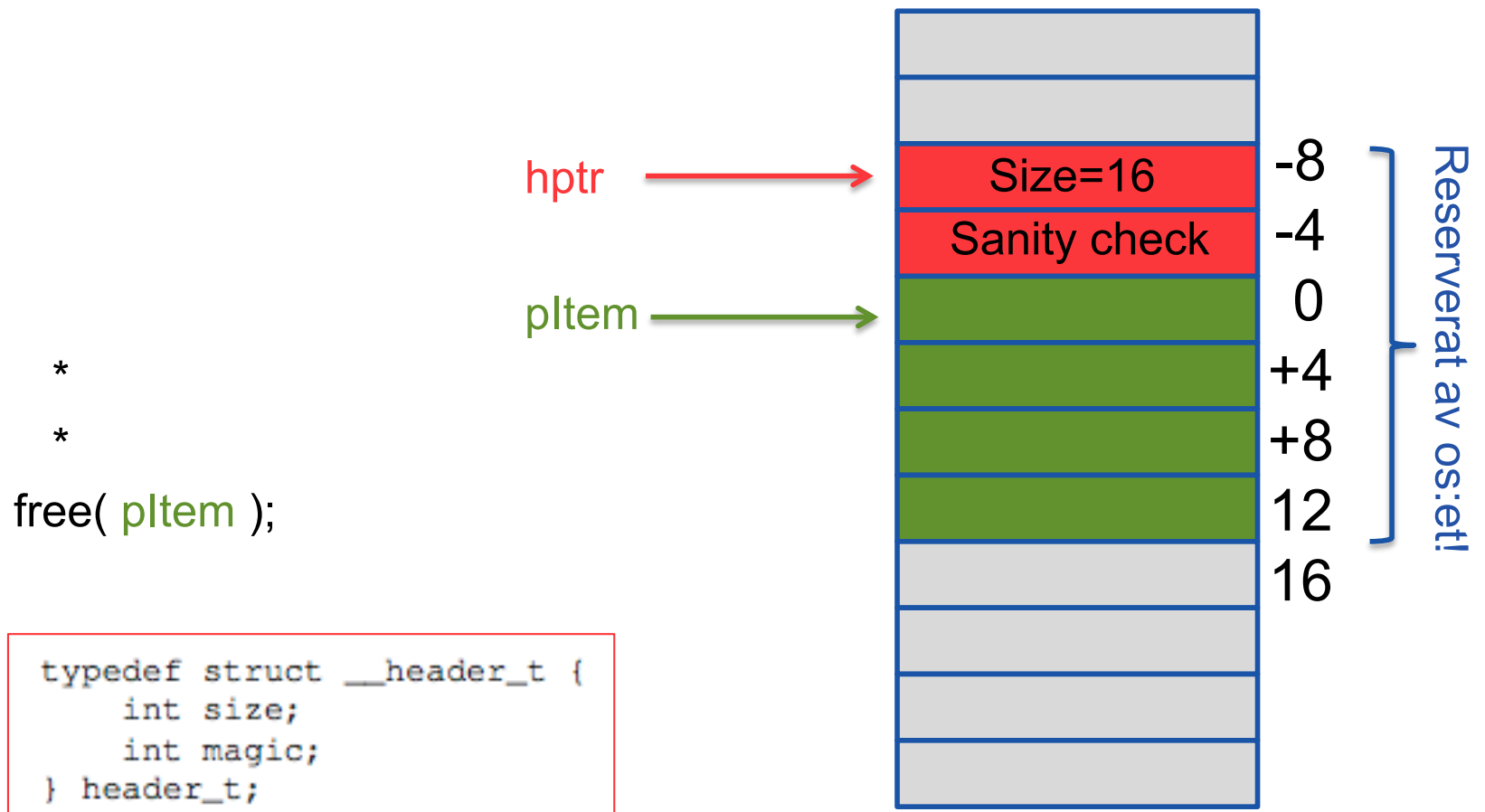
Hur vet OS:et hur många byte som returneras när free() bara har ett argument som är adressen till den första (enda?) byten i blocket?!

malloc() returnerar en pekare till första(/enda) byten som efterfrågats men gömmer på pekaren -4 & -8 typiskt information om tilldelat blocks storlek m.m. OS:et reserverar med andra ord 8 bytes mer än vad som efterfrågas så den kan hantera blocket när det lämnas tillbaks!

Spåra storleken på utdelat minne?!



Spåra storleken på utdelat minne?!



```

void free(void *ptr) {
    header_t *hptr = (void *)ptr - sizeof(header_t);
    ...
}
    
```

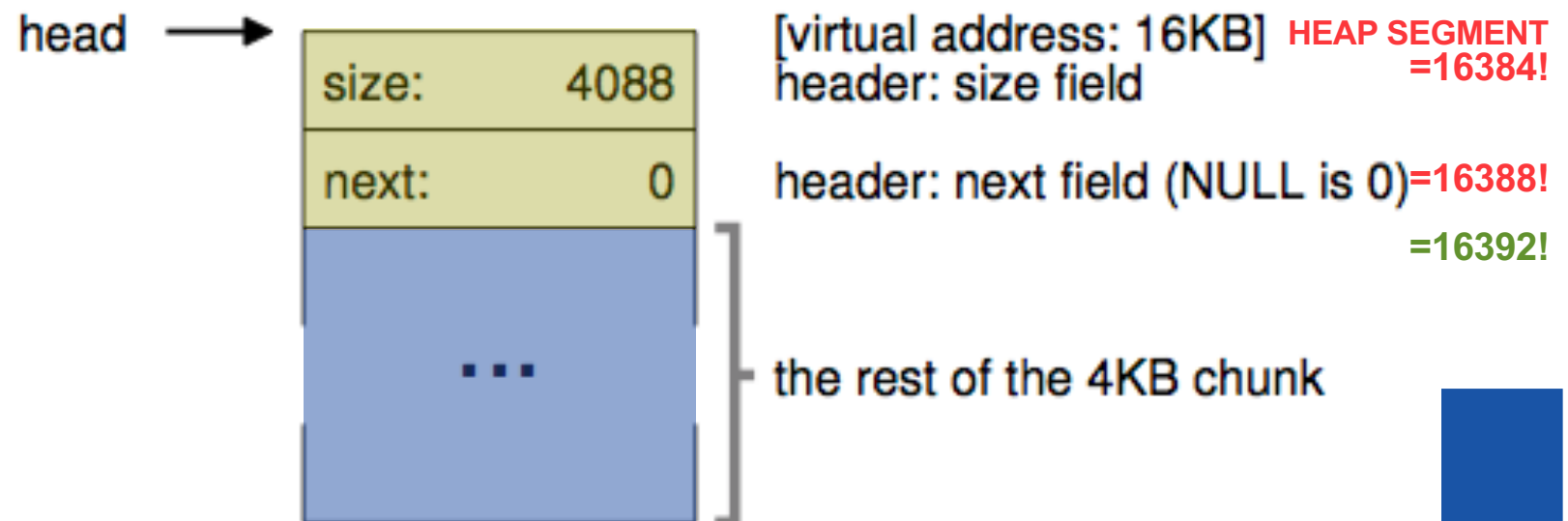



Hantera Free-list...

```
typedef struct __node_t {  
    int          size;  
    struct __node_t *next;  
} node_t;
```

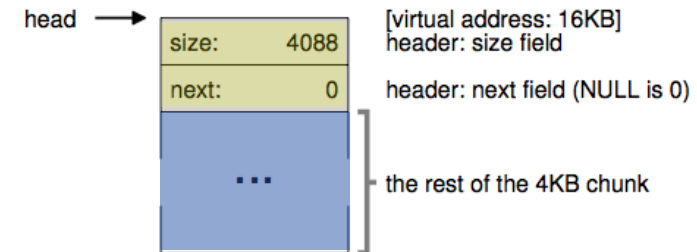
Tänk er att det finns en till struct som påminner om den som läggs före ett utdelat minnesblock, men där den andra medlemmen är en pekare som kan peka på någon av de två strukturerna – det kommer att bli en så kallad länkad lista!

Structen använd för att skapa os:ets första free-list som då ska innehålla allt tillgängligt heap-minne (kostar 2*4 byte)...





Hantera Free-list...



```
ptr=malloc(100);
```

Search free list...

...split (the only) free block...

(Ta 100+8 bytes från början)

...fixa "the still free" part:

size=4088-100-8

next=null (sista blocket)

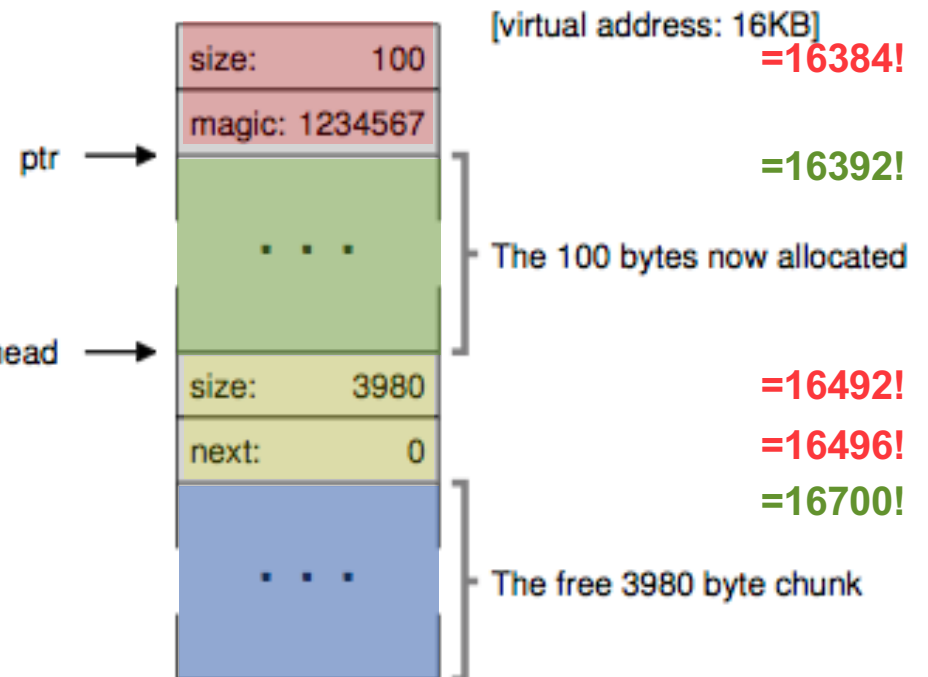
...fixa "tilldelat minne":

size=100

magic=sanity check #

...returnera adress head+8!

16392



OS:et kan komma fram till att den saknar heap-minne, men den har egentligen inte längre någon "koll" på det utlämnade blocket!



Hantera Free-list...

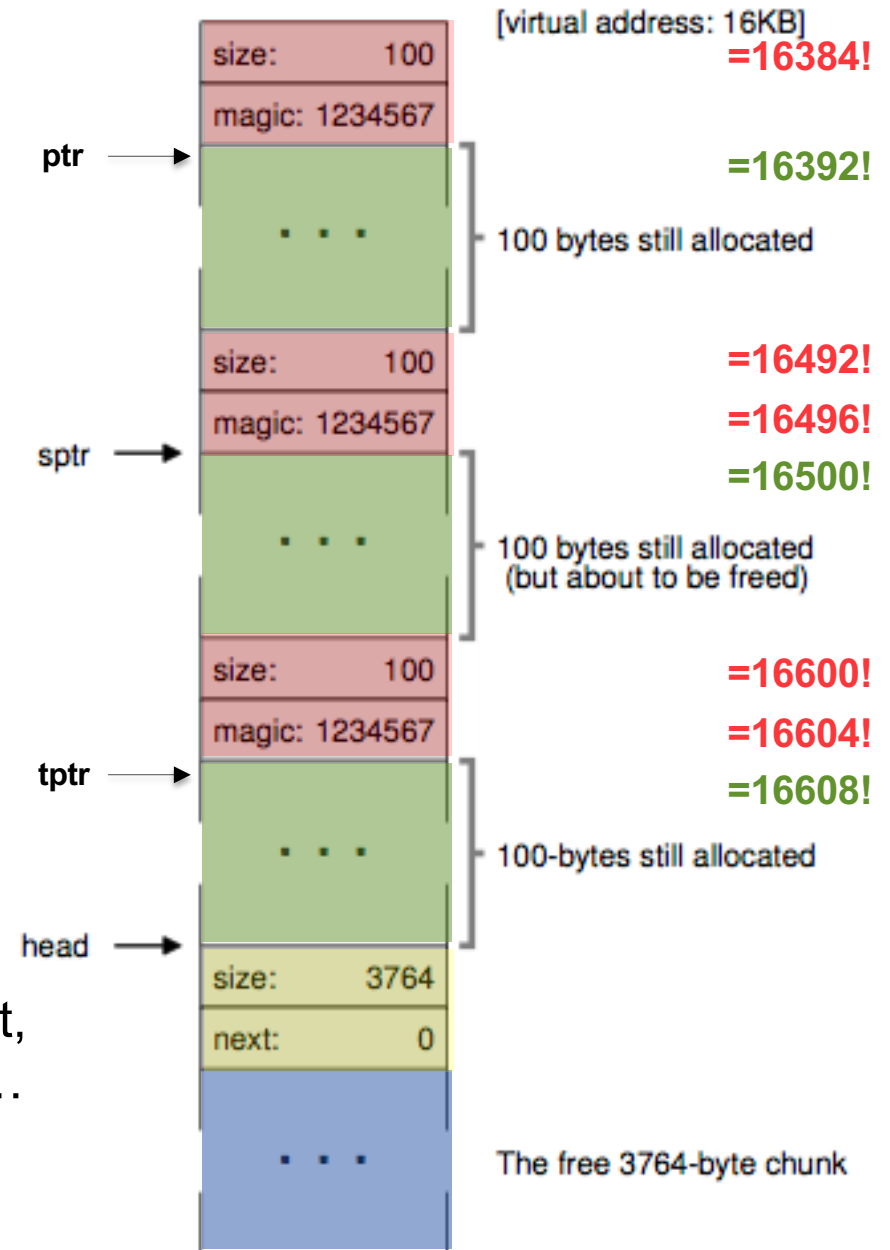
```
ptr=malloc(100);
```

Processen som kör behöver
2 till block om 100 byte...

```
sptr=malloc(100);
```

```
tptr=malloc(100);
```

...då borde heapen se ut så här:
OS:et känner till 3764 byte i freelist,
körande process äger 3*100 byte...



Vad händer nu om processen vill lämna tillbaka de 100 byte som processens sptr variabel pekar på: free(sptr)?



**Freelist är nu en länkad lista av den typen vi tittade på nyss...
Head -> 100/16708 -> 3764/NULL, men den är också fragmenterad!**



Hantera Free-list...

Lämnar processen tillbaks de andra 2
100 bytes-blocken blir det lätt så
här...

Head -> 100/16384 ->
100/16492 ->
100/16708 ->
3764/NULL

Vår free-funktion gör inte coalesce!

Det har visat sig att behovet av/möjlig-
heten att, enkelt göra coalesce beror
på hur vi delar ut minnet (split) så de
två delarna är sammankopplade!



(Uppenbart kan datorn "stanna upp" en stund, gå igen free-list, och slå samman block som ligger kant i kant = garbage collection, men det är sällan en bra väg...)



Tänkbara algoritmer: Best Fit!

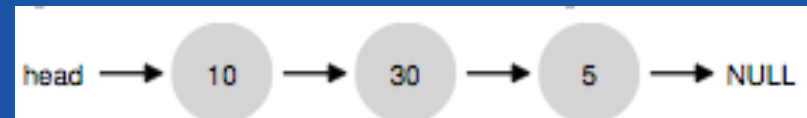
Sök igenom hela listan och välj, om möjligt, ett ledigt block av exakt rätt storlek – i annat fall ta så många byte som önskats från det lediga block som är närmast större.

+ Försöker minimera spill

- Hela listan måste sökas igenom, vilket tar tid
- Många små oanvändbara "spill"

(Söker igenom hela listan)

pBlock = malloc(7(+8));





Tänkbara algoritmer: Worst Fit!

Sök igenom hela listan och hitta det största block som finns kvar. Ta från det så många bytes som efterfrågas!

+ Försöker lämna användbara block kvar.

- Hela listan måste sökas igenom, vilket tar tid
- Har visat sig sämre än Best Fit i praktiken,
- Leder snabbt till fragmentering.

(Söker igenom hela listan)

```
pBlock = malloc(7(+8));
```





Tänkbara algoritmer: First Fit!

Sök igenom free listan tills ett tillräckligt stort block hittas.
Dela det vid behov och returnera önskat block.

- + Hela listan behöver inte sökas igenom
- Början av listan fylls snabbt av små block vilket ökar fragmentering/söktiden. (kan motverkas om free list hålls sorterad på något enkelt sätt.)

`pBlock = malloc(7(+8));`

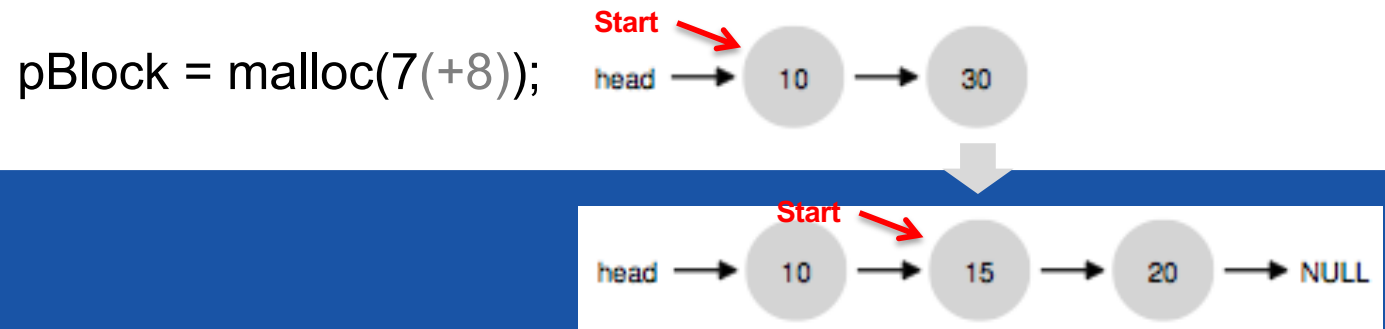




Tänkbara algoritmer: Next Fit!

Sök igenom free listan tills ett tillräckligt stort block hittas, men börja söka från det block som senast användes. Dela det vid behov och returnera önskat block. En extra pekare behövs med andra ord.

- + Hela listan behöver inte sökas igenom
- Har i praktiken visat sig obetydligt bättre än First Fit.





Sist: Segregated Lists – Buddy Allocation!

Det går att tillverka bättre free list hanterare än de vi sett...

...är det så att vissa blockstorlekar är mer efterfrågade än andra, vilket ibland går att förutse (interna os strukturer/ egen allokerare applikation), så kan de hanteras separat!

...allokeraren managerar då ett antal free list där alla utom en består av block av en fix storlek (lätta att hantera) samt en lista som tar hand om övriga förfrågningar!

...ett trick som dras till sin spets i Buddy metoden...



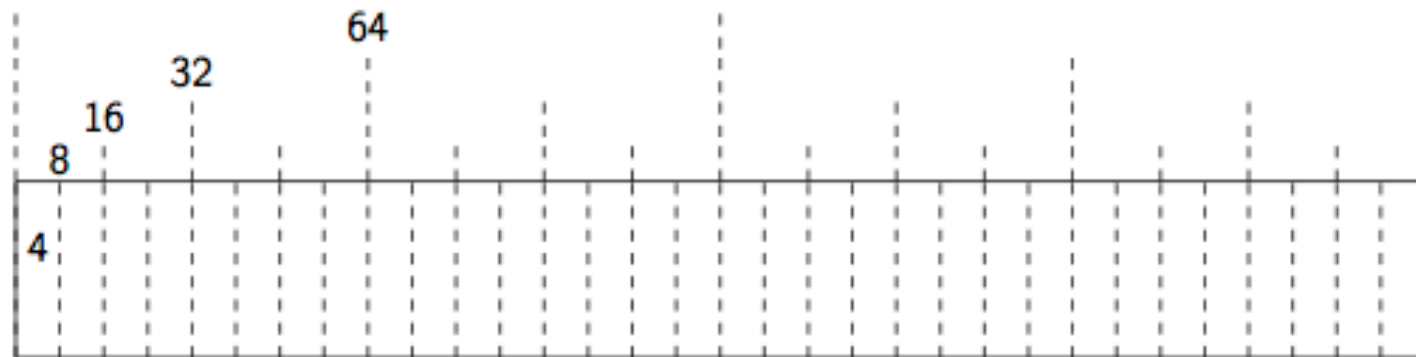
Sist: Segregated Lists – Buddy Allocation!

0 0000 F
0 0001 F
0 0010 F
0 0011 F

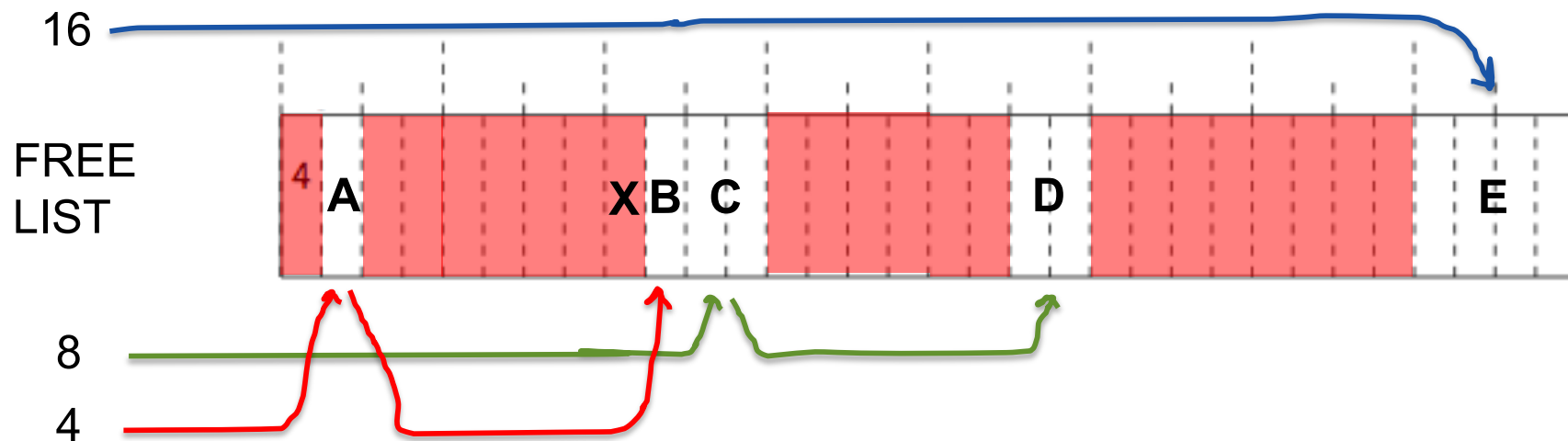
0 0100 F
0 0101 F
0 0110 F
0 0111 F

1 0000 F
1 0001 F
1 0010 F
1 0011 F

1 0100 F
1 0101 F
1 0110 F
1 0111 F



Sist: Segregated Lists – Buddy Allocation!



Väldigt enkelt att hantera malloc() & free()/coalescing!
Liten extern fragmentering, viss intern fragmentering.

(Sökt 12 får 16)



Om Heapen tar slut?!

```
#include <sys/mman.h>
```

```
void *mmap(void *addr,  
           size_t length,  
           int prot,  
           int flags,  
           int fd,  
           off_t offset);
```

`mmap()` creates a new mapping in the virtual address space of the calling process.

The length argument specifies the length of the mapping.

If `addr` is `NULL`, then the kernel chooses the address at which to create the mapping;

The `prot` argument describes the desired memory protection of the mapping..

`flags`, `fd` and `offset` for mapping of file in memory

Originally from 4.2BSD, default in OSX where `malloc()` uses `mmap()` to allocate memory.

(Linux använder också `brk()` and `sbrk()` för att öka ett programs heap)
(`brk` = break = vart programkoden slutar och heapen börjar)



Fördjupning Buddy-list implementering

Numreras noderna som i figuren kan en nods...

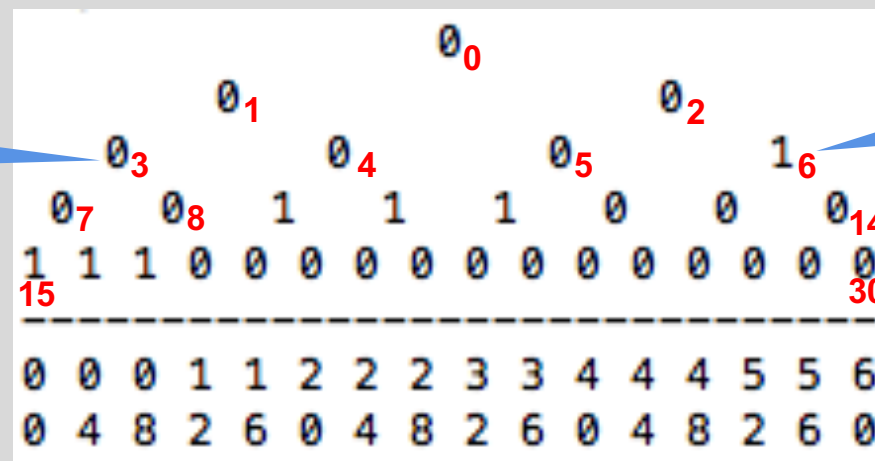
...”förälder” fås direkt via $\text{nodnummer} - 1/2$ &

...”barn” fås direkt via $\text{nodnummer} * 2 + 1$ (vänster) + 2 (höger)

Låt nodnummret indikera en bit i en 32-bits unsigned int...

...så kan hela heapen manageras via ett enda double word!

$3 * 2 + 1 = 7$
 $3 * 2 + 2 = 8$



Förälder
 $6 - 1/2 = 2$

X000 0000 0000 0011 1000 1110 0100 0000



Fördjupning Buddy-list implementering

```
// View - Pretty print of a BBT *****
void view(unsigned int aBBT){
    printf("%16d\n", aBBT&1);
    printf("%8d%16d\n", aBBT&1<<1?1:0, aBBT&1<<2?1:0);
    printf("%4d%8d%8d%8d\n", aBBT&1<<3?1:0, aBBT&1<<4?1:0,
        aBBT&1<<5?1:0, aBBT&1<<6?1:0);
    printf("%2d%4d%4d%4d%4d%4d%4d%4d\n",
        aBBT&1<<7?1:0, aBBT&1<<8?1:0,
        aBBT&1<<9?1:0, aBBT&1<<10?1:0,
        aBBT&1<<11?1:0, aBBT&1<<12?1:0,
        aBBT&1<<13?1:0, aBBT&1<<14?1:0);
    for (int i=15; i<31; i++) printf("%1d ", aBBT&1<<i?1:0);
    printf("\n-----\n");
    printf("0 0 0 1 1 2 2 2 3 3 4 4 4 5 5 6\n");
    printf("0 4 8 2 6 0 4 8 2 6 0 4 8 2 6 0\n");
}
```

Vi behöver operationer på bitnivå, något som c stödjer!



Fördjupning Buddy-list implementering

```
int mal(short size, unsigned int *aBBTp, short deep, short at){
    int addr=-1;
    if (size>(1<<(6-deep)) || (*aBBTp)&(1<<at))           // if (size>block or allocated)
                                                            // yes -> dead end, stop
        return -1;                                         // no -> continue
    else                                                    // if (size>block/2)
        if (size>(1<<(5-deep)))                            // yes -> truly free?
                                                            // yes -> allocate block!
            if (!add(*aBBTp,deep, at)) {                  // compute base addr
                                                            // and return it.
                (*aBBTp)+=(1<<at);
                while (at<15) at=at*2+1;
                return 4*(at-15);
            }
        else
            return -1;                                     // no -> dead end, stop
    else {
        if ((addr=mal(size, aBBTp, deep+1, at*2+1))<0)    // no -> try both children!
            addr=mal(size, aBBTp, deep+1, at*2+2);
        return addr;
    }
}
```

Systemprogrammering = effektiv är viktig = inte alltid lättläst...



Fördjupning Buddy-list implementering

```
// Add - Traverse and adds node value for start nod and each of its children ****
int add(unsigned int aBBT, short deep, short at){
    if (deep==4)
        return aBBT&1<<at?1:0;
    else
        return (aBBT&1<<at?1:0)+add(aBBT, deep+1, at*2+1)+add(aBBT, deep+1, at*2+2);
}
```

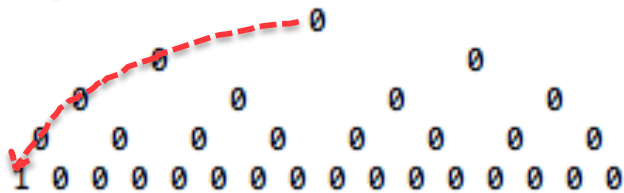
```
// Free - Returns a previously allocated block.....
void free(int addr, unsigned int *aBBTp){
    int at=(addr/4)+15; // Compute base leaf for the block...
    while (!((*aBBTp)&1<<at)) // ...travers BBT backwards until "1"...
        at=(at-1)/2;
    (*aBBTp)=(*aBBTp)^1<<at; // ...and deallocate the block (causing
    // an implicit coalescing of the memory!)
}
```

Systemprogrammering = effektiv är viktig = inte alltid lättläst...

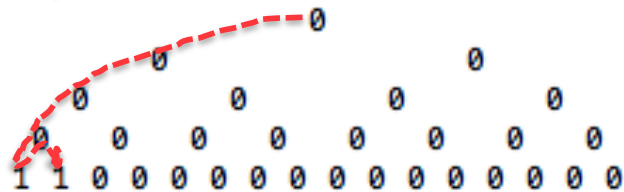
Buddy list allocation example 0.1 AC

Try mal(128) returns -1

Try mal(4) returns 0



Try mal(4) returns 4



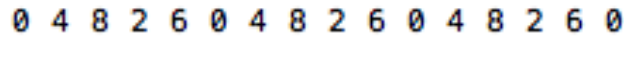
Try mal(4) returns 8



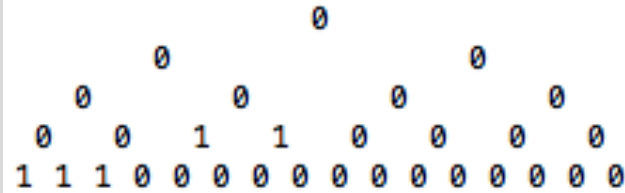
Try mal(8) returns 16



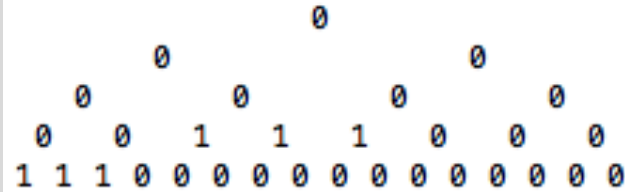
Try mal(8) returns 24



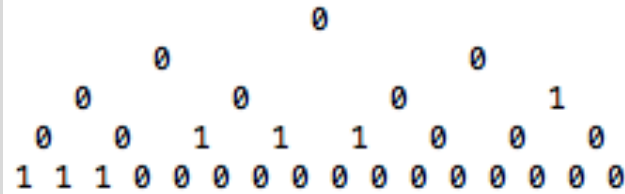
Try mal(8) returns 32



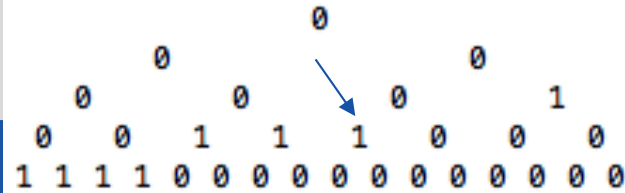
Try mal(8) returns 48



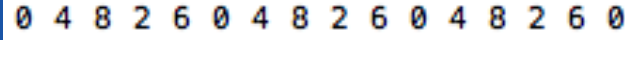
Try mal(16) returns 12



Try mal(4) returns -1

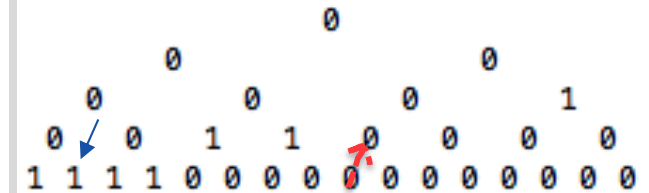


Try mal(4) returns -1

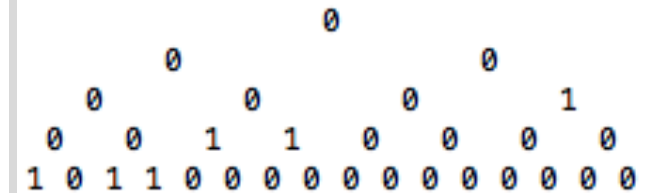


64 Byte Block
32 Byte Blocks
16 Byte Blocks
8 Byte Blocks
4 Byte Blocks

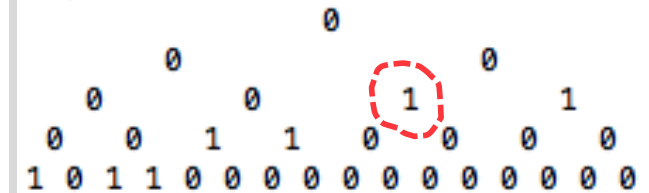
Free(32)



Free(4)



Try mal(16) returns 32



Try mal(16) returns -1



Done



Sammantaget...

1. Explicit mem man: the prg needs to explicitly free objects.
 - Used in C, C++ and most system level programming languages.
 - Pros: efficient usage of memory.
 - Cons: hard to find bugs when you don't do it right.
2. Implicit mem man: memory is freed by the system.
 - Managed by the runtime system i.e. a garbage collector (Java, Erlang, Python, ..) or by the compiler (Mercury, Rust ...).
 - Pros: much simpler and/or safer.
 - Cons: could result in runtime overhead and/or lack of control.



Nya/förstärkta hårdvarukrav

Hardware Requirements	Notes
Privileged mode	<i>Needed to prevent user-mode processes from executing privileged operations</i>
Base/bounds registers	<i>Need pair of registers per CPU to support address translation and bounds checks</i>
Ability to translate virtual addresses and check if within bounds	<i>Circuitry to do translations and check limits; in this case, quite simple</i>
Privileged instruction(s) to update base/bounds	<i>OS must be able to set these values before letting a user program run</i>
Privileged instruction(s) to register exception handlers	<i>OS must be able to tell hardware what code to run if exception occurs</i>
Ability to raise exceptions	<i>When processes try to access privileged instructions or out-of-bounds memory</i>

Figure 15.3: **Dynamic Relocation: Hardware Requirements**



Nya uppgifter för os:et

OS Requirements	Notes
Memory management	<i>Need to allocate memory for new processes; Reclaim memory from terminated processes; Generally manage memory via free list</i>
Base/bounds management	<i>Must set base/bounds properly upon context switch</i>
Exception handling	<i>Code to run when exceptions arise; likely action is to terminate offending process</i>

Figure 15.4: **Dynamic Relocation: Operating System Responsibilities**



Implikationer för boot-processen...

OS @ boot (kernel mode)	Hardware	(No Program Yet)
initialize trap table	remember addresses of...	
	system call handler	
	timer handler	
	illegal mem-access handler	
	illegal instruction handler	
start interrupt timer		
	start timer; interrupt after X ms	
initialize process table		
initialize free list		

Figure 15.5: Limited Direct Execution (Dynamic Relocation) @ Boot



Begrepp

Memory virtualization
Physical vs. Virtual Address
Transparent/Effective/Robust

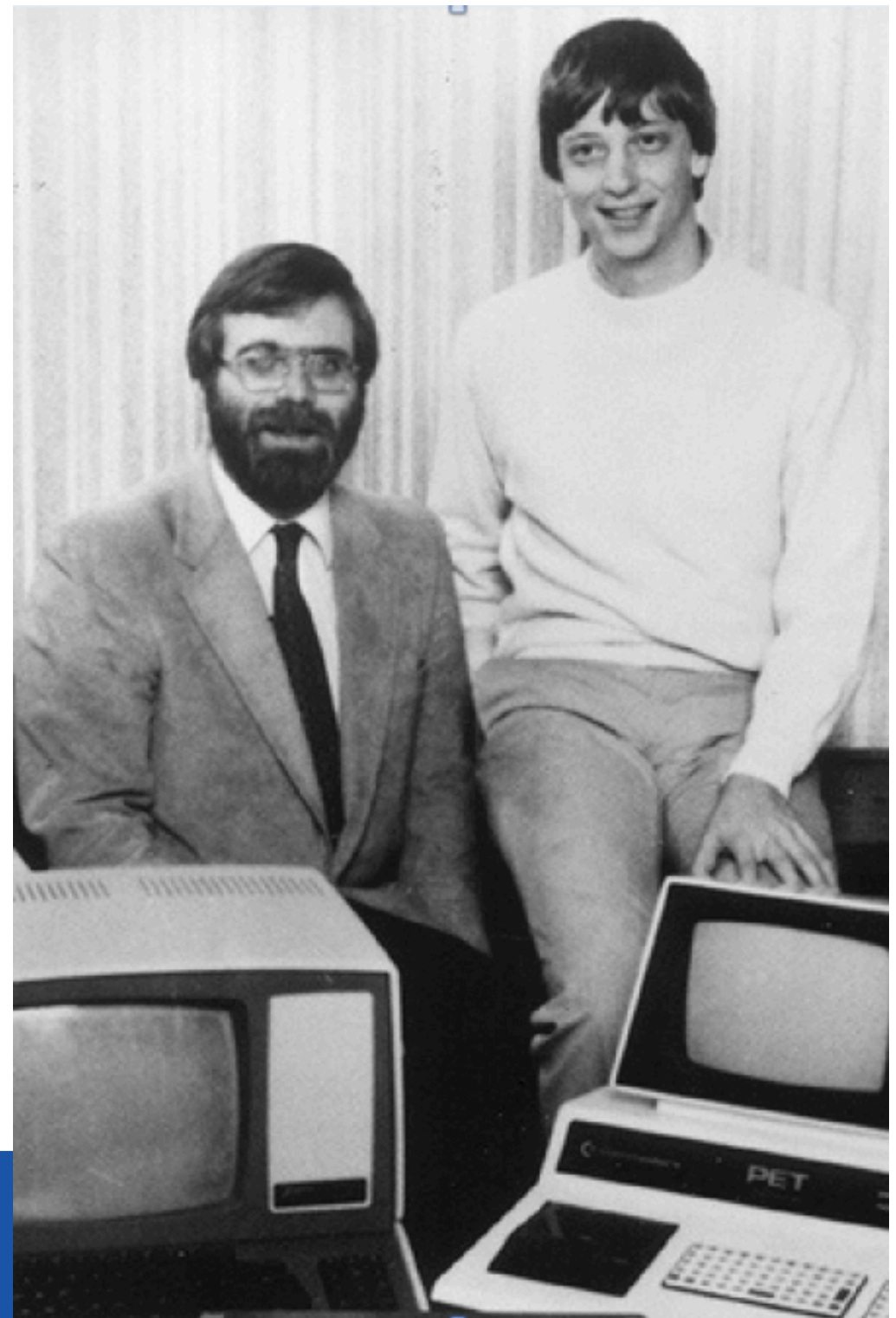
Emulation vs. static relocation
vs. dynamic relocation
Memory Management Unit (MMU)
Base & Bound Registers
Segmentation Model

Internal & External fragmentation
Free List, Splitting & Coalescing

Best, worst, first or next fit alg.
Garbage collection

Finding size of free() mem. Block?

Segregated lists
Buddy allocation





Historik

1810xx 0.1 AC Outline
1812xx 0.8 AC Draft
190110 1.0 AC Release 1
210409 1.1 AC Minor updates